

Update the extension logo

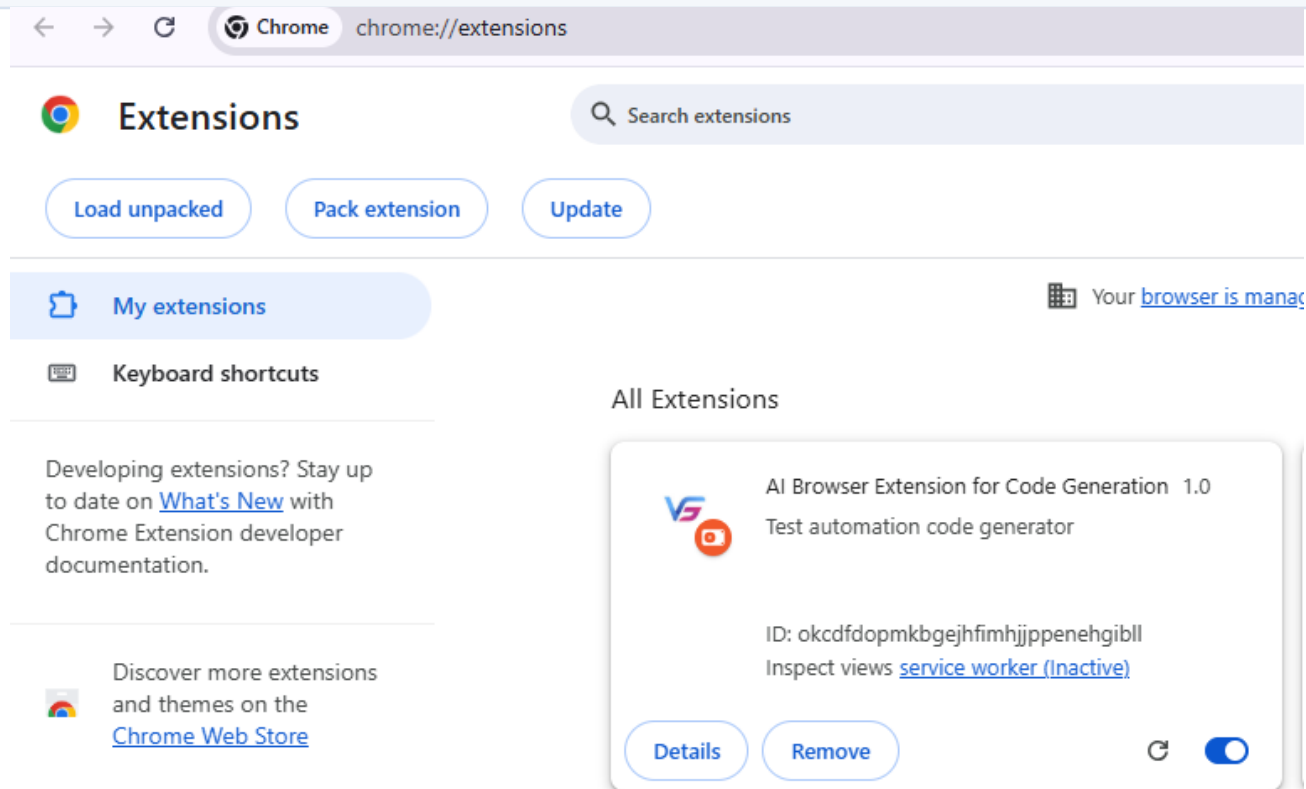
Required file changes followed by implementation evidence

1

Change the logo in the following files

- manifest.json
- appConfig.js

RESULT



2

Add "openai/gpt-oss-120b" to the existing list

- Update popup.js

IMPLEMENTATION NOTE

RESULT



Implement Playwright with TypeScript

Analysis-to-implementation workflow and generated-code evidence

3

Required workflow

- Use Ask Mode to analyze the existing project structure and identify all impacted files required for the Playwright with TypeScript implementation.
- Once the analysis is complete, switch to Agent Mode to implement the required changes.

GENERATED TYPESCRIPT RESULT

The screenshot displays a web browser window with the URL `leaf taps.com/opentaps/control/main`. The page shows the LeafTaps logo and a login form titled "LeafTaps Login" with fields for Username and Password, and a Login button. Overlaid on the right side of the browser is the "AI Browser Extension for Code Generation" interface. This interface has a "Code Generator" tab and a "Settings" tab. Under the "Code Generator" tab, there are two options: "Feature File" (selected) and "Page Class". A "Reset Chat" button is also present. The main area of the extension displays generated TypeScript code for a login form validation test.

```
import { test, expect } from '@playwright/test';

test('login form validation', async ({ page }) => {
  await page.goto('https://leaf taps.com/opentaps/control/main');
  await page.locator('#username').fill('testuser');
  await page.locator('#password').fill('testpassword');
  await page.locator('input[type="submit"]').click();
  await expect(page.locator('text="Login"')).toBeVisible();
  await expect(page.locator('#username')).toBeVisible();
  await expect(page.locator('#password')).toBeVisible();
});
```

EXPECTED CONFIGURATION

Supported Language: TypeScript | Browser Engine: Playwright

SETTINGS RESULT

The screenshot displays a web browser window with the address bar showing `leaf taps.com/opentaps/control/main`. The main content area is divided into two panels. The left panel, titled 'Testleaf Always Ahead', features a 'Leaf taps Login' section with input fields for 'Username' and 'Password', and a 'Login' button. The right panel, titled 'AI Browser Extension for Code Generation', shows the 'Settings' tab. The settings include: 'Supported Language' set to 'TypeScript', 'Browser Engine' set to 'Playwright', 'Select LLM Provider' set to 'Groq', 'Select Model' set to 'llama-3.3-70b-versatile', and a 'Groq API Key' field with a masked value.